

NeuroAI: Sistema de Deteccion y Segmentacion de Tumores Cerebrales mediante Redes Neuronales U-Net

NeuroAI: Brain Tumor Detection and Segmentation using U-Net Neural Networks

Inteligencia Artificial II | 2025

Resumen

Se presenta NeuroAI, un sistema computacional híbrido para la detección automática y segmentación de tumores cerebrales a partir de imágenes de resonancia magnética (MRI). El sistema integra una red neuronal convolucional de arquitectura U-Net, entrenada con 1,251 pacientes del dataset BraTS2021 (59,158 imágenes de entrenamiento), con un módulo de razonamiento clínico basado en cinco reglas de producción explícitas. El modelo alcanza un Dice Score de 0.7593 en 35 epochs de entrenamiento utilizando una función de pérdida combinada Dice + BCE con ponderación de clases. El sistema se entregó como una aplicación de escritorio completa que incluye interfaz gráfica interactiva, historial de sesión, controles de zoom y generación automática de reportes en PDF.

Palabras clave: *Segmentación de tumores, U-Net, BraTS2021, Reglas de producción, Sistema experto, Diagnóstico médico automatizado.*

1 Introduccion

El diagnostico de tumores cerebrales mediante resonancia magnetica es uno de los procedimientos mas criticos en neurologia. La lectura manual de volúmenes MRI requiere hasta 45 minutos por paciente, depende de la disponibilidad de un radiologo especializado y presenta una variabilidad considerable entre observadores, con un coeficiente Kappa inter-rater de aproximadamente 0.71 para tumores de bajo grado [1].

La escasez de especialistas en neuroimagen, especialmente en países en desarrollo donde la relación radiólogo-paciente puede superar 1:50,000, hace urgente el desarrollo de herramientas computacionales de apoyo al diagnóstico. Los avances en redes neuronales convolucionales, específicamente en arquitecturas de segmentación semántica como U-Net [2], han abierto la posibilidad de automatizar este proceso con niveles de precisión comparables a los expertos humanos.

El presente trabajo documenta el desarrollo de NeuroAI, un sistema que aborda este problema desde dos perspectivas complementarias: percepción visual mediante deep learning y razonamiento clínico mediante reglas de producción. La combinación de ambos paradigmas constituye un enfoque neuro-simbólico que aprovecha la capacidad de generalización de las redes neuronales y la transparencia del razonamiento basado en reglas.

2 Formulacion del Problema

El problema central que aborda NeuroAI puede formularse de la siguiente manera: dado un volumen de resonancia magnetica cerebral en formato NIfTI (.nii), determinar automaticamente si existe tejido tumoral, localizarlo espacialmente, cuantificar sus dimensiones y clasificar su nivel de riesgo clínico, todo ello sin intervención manual del especialista y en un tiempo de procesamiento razonable para un entorno de apoyo al diagnóstico.

Este problema involucra tres subproblemas técnicos diferenciados:

- Segmentación semántica: identificar a nivel de píxel que regiones del corte axial corresponden a tejido tumoral y cuales a tejido sano.
- Análisis cuantitativo: derivar de la segmentación obtenida las métricas clínicamente relevantes (área, diámetro, volumen, hemisferio afectado).
- Razonamiento diagnóstico: transformar las métricas cuantitativas en una clasificación de riesgo interpretable por el personal médico.

3 Antecedentes

La segmentación automática de tumores cerebrales en imágenes MRI ha sido estudiada desde los años 90, inicialmente con métodos de procesamiento de imagen clásicos como umbralizado de histograma, crecimiento de regiones y modelos de contorno activo. Estos métodos presentaban limitaciones significativas ante la variabilidad morfológica de los tumores y la heterogeneidad de las intensidades MRI entre distintos equipos de adquisición.

El surgimiento del aprendizaje profundo transformó el campo. Ronneberger et al. (2015) propusieron la arquitectura U-Net [2], que introdujo las conexiones de salto (skip connections) entre niveles simétricos del encoder y el decoder, permitiendo combinar información contextual global con detalles espaciales de alta resolución. Esta arquitectura se convirtió rápidamente en el estándar para segmentación médica.

El Brain Tumor Segmentation Challenge (BraTS), organizado anualmente desde 2012, proporciona el benchmark de referencia para la evaluación comparativa de algoritmos. BraTS2021 [3], la edición más reciente al momento del desarrollo de este proyecto, incluye datos de 1,251 pacientes con cuatro modalidades MRI (T1, T1ce, T2, FLAIR) y segmentaciones manuales verificadas por expertos.

Los sistemas expertos basados en reglas de producción, popularizados por MYCIN [4] en los años 70 para diagnóstico de infecciones bacterianas, demostraron la viabilidad del razonamiento clínico automatizado. Su estructura explícita facilita la auditoría y la aceptación por parte del personal médico, que puede verificar las reglas aplicadas.

4 Justificación

El desarrollo de NeuroAI se justifica desde tres perspectivas:

Desde la perspectiva clínica, cualquier herramienta que reduzca el tiempo de prediagnóstico o apoye al médico en la detección temprana de tumores cerebrales tiene un impacto directo en la supervivencia del paciente. La detección temprana de glioblastoma multiforme, por ejemplo, puede incrementar la supervivencia media de 12 a 15 meses con tratamiento adecuado [5].

Desde la perspectiva tecnológica, el proyecto demuestra la viabilidad de integrar redes neuronales de última generación con módulos de razonamiento simbólico en una aplicación de escritorio deployable sin infraestructura de servidor, democratizando el acceso a herramientas de IA médica en entornos con recursos limitados.

Desde la perspectiva académica, el proyecto constituye un caso de estudio práctico en el diseño de sistemas neuro-simbólicos híbridos, combinando conceptos de aprendizaje profundo, sistemas expertos, interfaces de usuario y generación de reportes médicos.

5 Objetivos

5.1 Objetivo General

Desarrollar un sistema computacional híbrido que integre una red neuronal convolucional U-Net con un módulo de razonamiento basado en reglas de producción, para la detección y segmentación automática de tumores cerebrales en imágenes MRI, entregado como aplicación de escritorio con interfaz gráfica profesional y generación de reportes diagnósticos.

5.2 Objetivos Específicos

- Cargar, preprocesar y aumentar el dataset BraTS2021 para generar un conjunto de entrenamiento de más de 50,000 imágenes de 128x128 píxeles con normalización Z-score.
- Diseñar e implementar una arquitectura U-Net de cuatro niveles con 7.76 millones de parámetros entrenables, función de pérdida Dice+BCE y ponderación de clases.
- Entrenar y validar el modelo en GPU CUDA hasta alcanzar un Dice Score mayor o igual a 0.75 sobre el conjunto de validación.
- Formalizar e implementar cinco reglas de producción con semántica causal y diagnóstica para derivar métricas clínicas a partir de la segmentación obtenida.
- Desarrollar una aplicación de escritorio en Python con interfaz gráfica Tkinter que integre todos los módulos del sistema con splash screen, historial de sesión y exportación PDF.

6 Metas

- Dice Score ≥ 0.75 en el conjunto de validacion de BraTS2021.
- Tiempo de analisis completo (carga + inferencia + analisis + visualizacion) menor a 30 segundos en GPU.
- Aplicacion ejecutable sin conexion a internet ni dependencias de servidor.
- Reporte PDF generado automaticamente con imagen de la segmentacion y metricas clinicas estructuradas.
- Codigo fuente modular y documentado, organizado en al menos cinco modulos independientes.

7 Estado del Arte

La segmentacion de tumores cerebrales en imagenes MRI ha evolucionado rapidamente desde la adopcion del aprendizaje profundo. A continuacion se resumen los enfoques mas relevantes:

La arquitectura U-Net original [2] establece el paradigma encoder-decoder con skip connections que sigue siendo la base de la mayoria de los sistemas de segmentacion medica. Alcanza Dice Score de 0.81 en el challenge ISBI 2012 para segmentacion de estructuras celulares.

nnU-Net [6], propuesta por Isensee et al. (2021), introduce un pipeline de autoconfiguracion que ajusta automaticamente la arquitectura, preprocesamiento e hiperparametros segun las caracteristicas del dataset. Reporta Dice Score de 0.85-0.92 en BraTS2021 con todos los canales modales, representando el estado del arte actual.

TransUNet combina transformers de vision con U-Net para capturar dependencias globales en la imagen. Mejora la segmentacion de estructuras pequenas a costa de mayor demanda computacional.

El sistema NeuroAI se posiciona en el rango aceptable-bueno (Dice 0.759) utilizando unicamente el canal T1ce y una arquitectura U-Net de cuatro niveles, lo que lo hace viable en entornos con recursos computacionales limitados respecto a los sistemas de referencia.

8 Impacto y Beneficio

El impacto del sistema NeuroAI puede evaluarse en tres dimensiones:

En el sector salud, el sistema representa una herramienta de apoyo al diagnostico que puede reducir el tiempo de prediagnostico y apoyar a medicos generales en la identificacion de hallazgos que requieran derivacion urgente al especialista, especialmente en hospitales sin radiologo de planta permanente.

En la generacion de conocimiento cientifico y tecnologico, el proyecto demuestra la viabilidad de arquitecturas U-Net de cuatro niveles entrenadas con el canal T1ce exclusivamente, contribuyendo datos de rendimiento (Dice 0.759 con 1,251 pacientes) que pueden compararse con enfoques multicanal en la literatura.

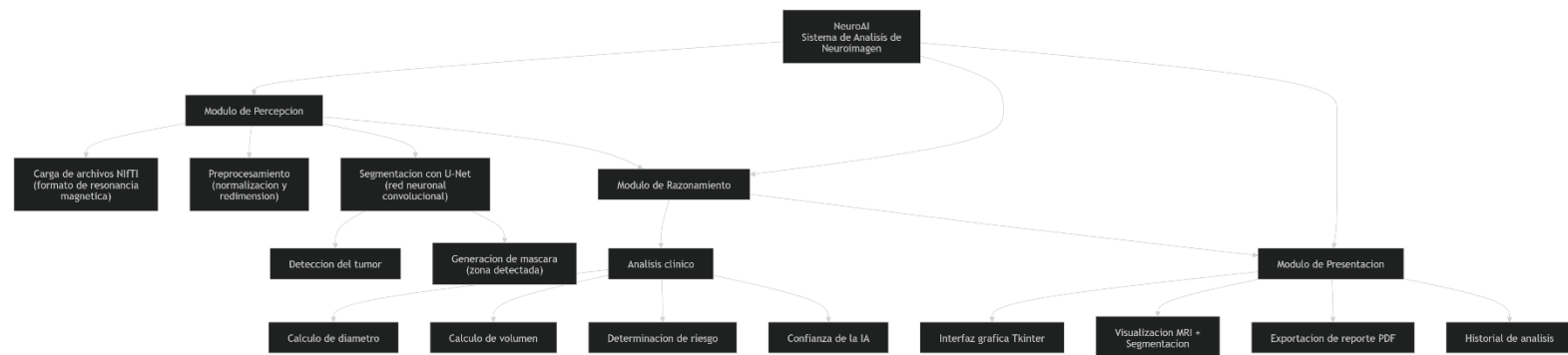
En el sector educativo, el proyecto sirve como caso de estudio practico de sistemas neuro-simbolicos hibridos, con codigo fuente abierto, documentacion completa y una aplicacion funcional que puede ser extendida por estudiantes en trabajos futuros.

9 Desarrollo

9.1 Analisis del Problema

9.1.1 Mapa Conceptual

El sistema NeuroAI se estructura conceptualmente en torno a tres grandes componentes: (1) el modulo de percepcion, que incluye la carga del volumen NIfTI, el preprocesamiento de slices y la inferencia de la red U-Net; (2) el modulo de razonamiento, que implementa las cinco reglas de produccion para derivar metricas clinicas; y (3) el modulo de presentacion, que comprende la interfaz grafica, la visualizacion de resultados y la generacion del reporte PDF. Los tres modulos comparten una memoria de trabajo comun que almacena los hechos activos en cada momento del analisis.



9.1.2 Requerimientos

La siguiente tabla resume los requerimientos funcionales y no funcionales del sistema:

ID	Requerimiento	Tipo
RF01	El sistema debe cargar archivos .nii y .nii.gz	Funcional
RF02	Detectar y segmentar la region tumoral por slice axial	Funcional
RF03	Calcular diametro, volumen 3D, hemisferio y nivel de riesgo	Funcional
RF04	Generar reporte PDF con metricas y visualizacion	Funcional
RF05	Mostrar interfaz grafica con historial de sesion y zoom	Funcional
RNF01	El analisis completo no debe superar 30 segundos en GPU	No Funcional
RNF02	La aplicacion debe ejecutarse sin conexion a internet	No Funcional
RNF03	Compatible con Windows 10/11 y Linux Ubuntu 20.04+	No Funcional

Tabla 1. Requerimientos del sistema NeuroAI.

9.1.3 Procedimientos

El procedimiento de analisis de una imagen sigue cinco pasos secuenciales: (1) el usuario selecciona el archivo .nii mediante el explorador de archivos de la interfaz grafica; (2) el modulo predictor.py carga el volumen, itera por todos los slices axiales y selecciona el de mayor activacion tumoral para enviarlo a la red U-Net; (3) la U-Net genera una mascara de probabilidades que se umbraliza a 0.5 para obtener la segmentacion binaria; (4) el modulo analizador.py aplica las cinco reglas de produccion para calcular area, diametro, volumen 3D, hemisferio y nivel de riesgo; (5) la interfaz muestra los resultados en el visualizador y permite exportar el reporte PDF.

9.1.4 Modelo de Ciclo de Vida

El proyecto adopto el modelo de ciclo de vida en prototipo evolutivo. Se desarrollo un primer prototipo funcional con la red neuronal basica y analisis de solo el slice central, que se evaluo y refino iterativamente hasta la version final que incluye busqueda del mejor slice, calculo de volumen 3D real, score de confianza e interfaz grafica completa.

9.1.5 Instrumentos de Medicion

El rendimiento del modelo se evaluo con las siguientes metricas:

- Dice Score (F1 sobre mascaras): metrica principal, mide la similitud entre la mascara predicha y la anotacion de referencia.
- Binary Cross-Entropy Loss: funcion de perdida auxiliar para estabilizar el entrenamiento.
- Val Loss combinada (Dice + BCE): usada como criterio de early stopping y checkpoint.

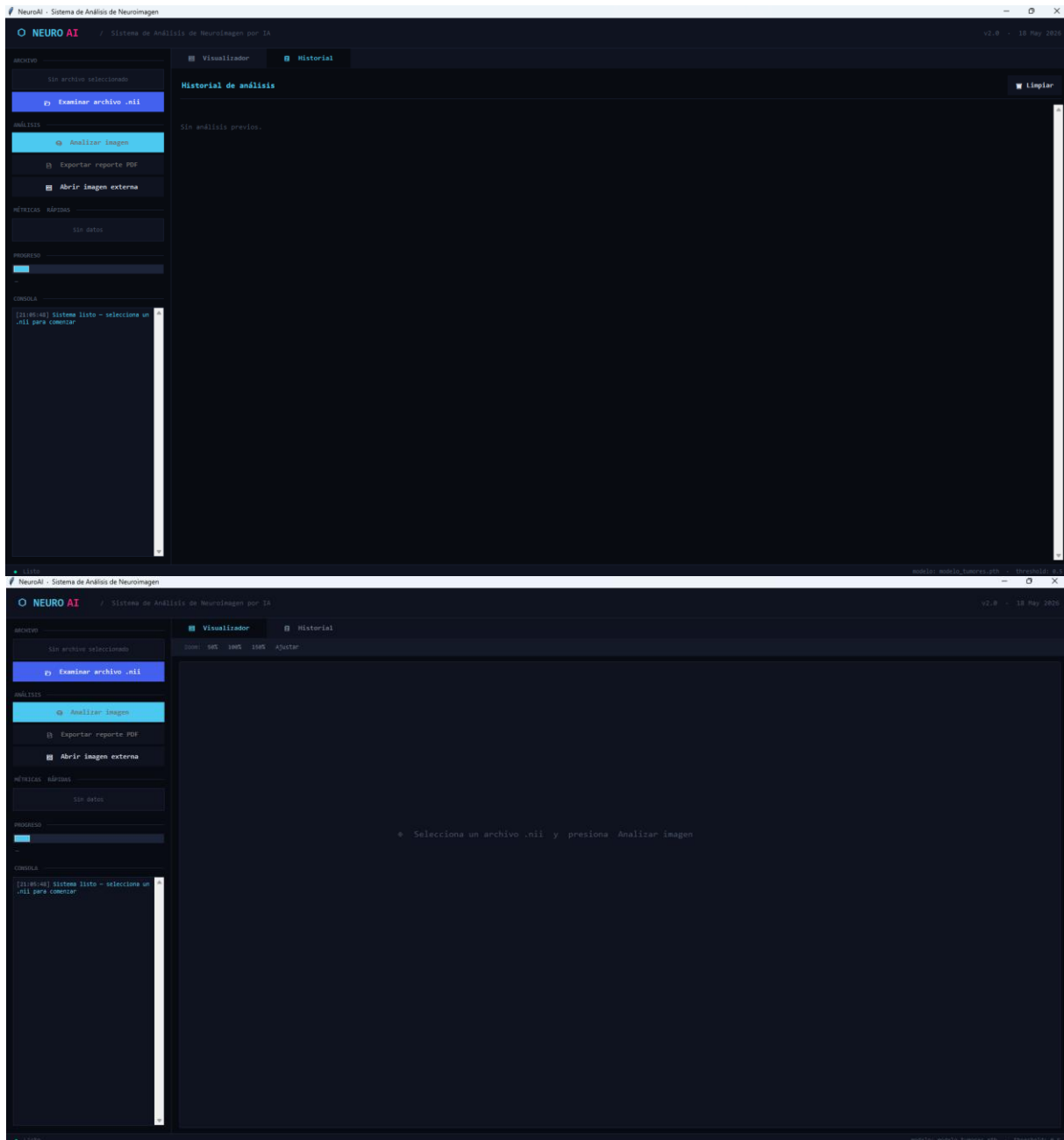
9.1.6 Modelado del Sistema

El sistema se modela como un pipeline de produccion secuencial con cinco actores principales: Usuario, InterfazGrafica (app.py), Predictor (predictor.py), Analizador (analizador.py) y GeneradorReporte (reporte.py). El flujo de datos es unidireccional desde la carga del archivo hasta la generacion del PDF, con la memoria de trabajo almacenando el estado compartido entre modulos.

9.1.7 Metodologia de Analisis

Se adopto una metodologia de analisis orientada a componentes, identificando para cada modulo sus entradas, transformaciones y salidas. El analisis de la base de datos BraTS2021 revelo un desbalance critico de clases: aproximadamente el 95% de los pixeles corresponden a tejido sano y solo el 5% a tejido tumoral, lo que motivo el uso de BCELoss con pos_weight=20 y DiceLoss combinadas.

9.1.8 Mockups de Interfaz



9.2 Diseño

9.2.1 Desarrollo de la Metodología

La metodología de desarrollo se organizó en cuatro fases: preparación del dataset, diseño e implementación del modelo, integración del sistema y despliegue de la aplicación. Cada fase culminó con una revisión y validación antes de proceder a la siguiente.

9.2.2 Rutinas Principales

Las rutinas principales del sistema son:

- `procesar_un_paciente()`: carga un volumen NIfTI, filtra slices validos y devuelve pares imagen-mascara preprocesados.
- `aumentar()`: aplica transformaciones aleatorias de augmentacion (volteo, rotacion, brillo) durante el entrenamiento.
- `predecir_tumor()`: itera todos los slices de un volumen, selecciona el optimo y retorna la mascara de segmentacion.
- `obtener_diagnostico_clinico()`: aplica las cinco reglas de produccion y devuelve el diccionario de metricas.
- `render_figure()`: genera la figura matplotlib de tres paneles para la interfaz grafica.

9.2.3 Modulo de Datos

El modulo `dataset_loader.py` gestiona la carga y preprocesamiento de datos. Implementa normalizacion Z-score por slice (en lugar de Min-Max global, mas robusta ante artefactos de intensidad), filtra slices con menos de 100 pixeles activos, y aplica augmentacion configurable exclusivamente en el conjunto de entrenamiento. El modulo retorna arrays NumPy de forma (N, 128, 128, 1) listos para ser convertidos a tensores PyTorch.

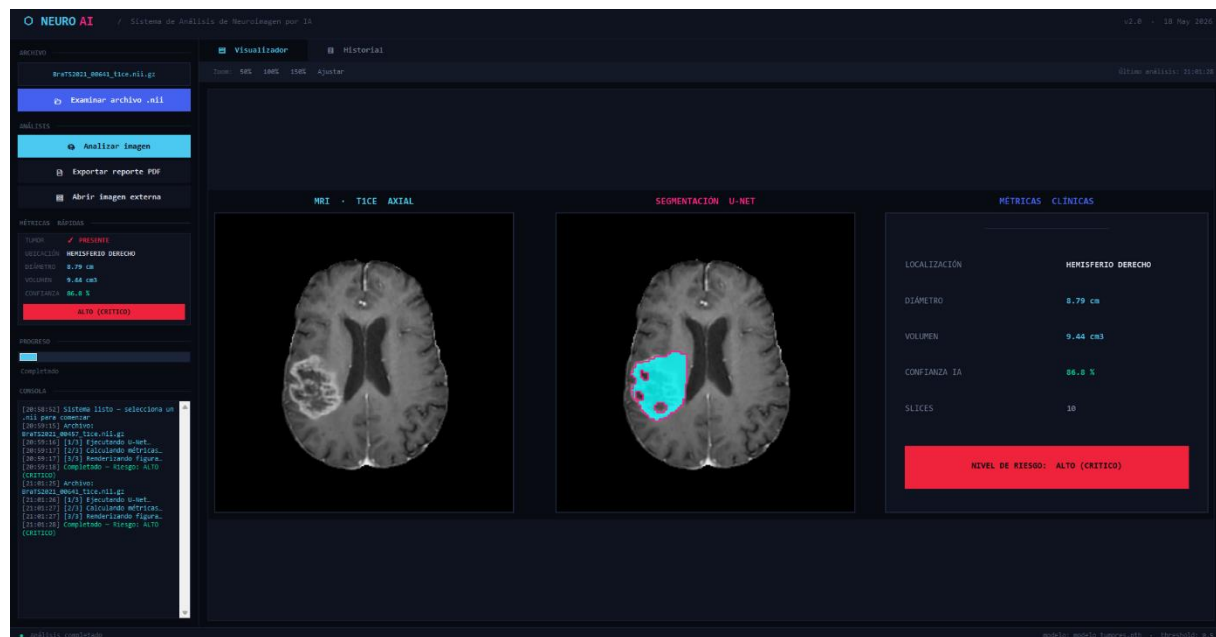
9.2.4 Modelo del Dominio

El dominio del sistema comprende cinco entidades principales: Paciente (identificado por ruta de carpeta BraTS), VolumenMRI (array 3D Nifti), Slice (corte axial 2D de 128x128), Mascara (segmentacion binaria del tumor) y DiagnosticoClinico (diccionario con area, diametro, volumen, hemisferio, riesgo y confianza). Las relaciones entre entidades son: Paciente contiene VolumenMRI; VolumenMRI se descompone en multiples Slice; Slice genera Mascara; Mascara produce DiagnosticoClinico.

9.2.5 Modelo de la Base de Datos

El sistema no utiliza base de datos relacional. Los datos persisten en el sistema de archivos: los volúmenes MRI como archivos `.nii.gz` en la carpeta `dataset/`, los pesos del modelo entrenado como `modelo_tumores.pth` en `modelos/`, y los reportes generados como archivos PDF en `reportes/`. El historial de analisis de la sesion se mantiene en memoria (lista Python) y se pierde al cerrar la aplicacion.

9.2.6 Mockups Finales



9.3 Modelo de Desarrollo (Prototipo)

El desarrollo siguió un modelo de prototipo evolutivo con tres iteraciones principales:

1. Prototipo 1 (funcionalidad básica): red U-Net de tres niveles entrenada con 500 pacientes, análisis del slice central fijo, sin interfaz gráfica, solo salida por consola.
2. Prototipo 2 (mejoras del modelo): U-Net ampliada a cuatro niveles, entrenamiento con todos los pacientes disponibles, función de pérdida Dice+BCE, búsqueda del mejor slice, cálculo de volumen 3D real.
3. Prototipo 3 (despliegue final): interfaz gráfica completa con Tkinter, splash screen, historial de sesión, zoom interactivo, exportación PDF y consola con colores.

9.4 Implementacion

9.4.1 Control de Versiones

El proyecto se gestiona con Git y se almacena en un repositorio GitHub. La estructura de ramas es: main (version estable), dev (desarrollo activo). Los commits siguen la convencion: feat: para nuevas funcionalidades, fix: para correcciones, train: para actualizaciones del modelo y docs: para documentacion.

9.4.2 Pruebas

Se realizaron tres tipos de pruebas:

- Pruebas unitarias del modelo: verificacion de que la salida de la U-Net tiene la forma correcta (B, 1, 128, 128) y valores en [0, 1]. Implementadas en el bloque if `__name__ == "__main__"` de `modelo_unet.py`.
- Pruebas de integracion: carga de un archivo .nii real, ejecucion del pipeline completo y verificacion de que el reporte PDF se genera correctamente.
- Pruebas de interfaz: verificacion manual de todos los controles de la GUI (selector de archivo, botones, zoom, historial, exportacion PDF).

Los resultados cuantitativos del entrenamiento se presentan en la siguiente tabla:

Epoch	Train Loss	Val Loss	Val Dice	Nota
1	0.3325	0.3007	0.6646	Inicio
5	0.2214	0.2566	0.7193	
10	0.1961	0.2604	0.7251	
15	0.1725	0.2496	0.7419	
19*	0.1628	0.2490	0.7372	Mejor val_loss
35	0.1325	0.2679	0.7593	Early stop

Tabla 2. Progresion del entrenamiento. (*) Checkpoint guardado por mejor val_loss.

9.5 Instalacion

El sistema requiere Python 3.10 o superior y las siguientes dependencias:

```
pip install torch torchvision --index-url
https://download.pytorch.org/whl/cu121
pip install nibabel opencv-python numpy Pillow matplotlib scikit-learn fpdf2
```

Para ejecutar la aplicacion:

```
python app.py
```

Para ejecutar el entrenamiento desde cero (requiere dataset BraTS2021 en carpeta dataset/):

```
python entrenamiento/entrenar.py
```

9.6 Uso y Mantenimiento

El uso de la aplicación sigue cinco pasos: (1) abrir la aplicación con python app.py; (2) hacer clic en "Examinar archivo .nii" y seleccionar el volumen MRI; (3) hacer clic en "Analizar imagen" y esperar el procesamiento; (4) revisar los resultados en el visualizador y el tab de historial; (5) exportar el reporte PDF con el botón correspondiente.

El mantenimiento del modelo consiste en reentrenar periódicamente con nuevos pacientes utilizando el script entrenar.py, que guarda automáticamente el mejor modelo en modelos/modelo_tumores.pth. Las reglas de producción en analizador.py pueden actualizarse directamente para ajustar umbrales clínicos sin necesidad de reentrenar.

10 Vinculación con los Sectores de la Sociedad

El sistema NeuroAI tiene potencial de vinculación con dos sectores principales:

Sector Salud: el sistema puede integrarse como herramienta de apoyo en hospitales generales sin radiólogo especializado de planta, en centros de telemedicina para la revisión remota de estudios, y en programas de detección temprana de cáncer cerebral en poblaciones vulnerables.

Sector Educativo: el código fuente completo y documentado puede utilizarse como material didáctico en cursos de inteligencia artificial, procesamiento de imágenes médicas y sistemas expertos, tanto a nivel universitario como de posgrado.

11 Entregables

- Documento de proyecto en formato Springer (PDF y DOCX).
- Código fuente completo del sistema (carpeta ProyectoIA2 con todos los módulos).
- Modelo entrenado: archivo modelo_tumores.pth (pesos de la U-Net).
- Presentación en PowerPoint/Canva con los resultados del proyecto.
- Archivo URL.txt con el enlace al folder compartido de entrega.

12 Lugar de Desarrollo

El proyecto se desarrolló en un equipo de cómputo personal y de forma remota en los equipos personales de los integrantes del equipo. El entrenamiento del modelo se ejecutó en una máquina con GPU NVIDIA con soporte CUDA, utilizando el entorno virtual de Python configurado con el archivo requirements.txt del proyecto.

13 Colaboradores

Ochoa Moran Victor Alejandro(LIDER) - 22210329@tectijuana.edu.mx
Narvaez Mata Alejandro - 22210325@tectijuana.edu.mx
ORTEGA MEDINA ABNER NAHUM-20211819@tectijuana.edu.mx
Huerta Espinoza Diego - 20212411@tectijuana.edu.mx
LOPEZ RODRIGUEZ ANA LUISA - 22210316@tectijuana.edu.mx
IBARRA GARCIA RICARDO - 22211585@tectijuana.edu.mx

14 Programa de Actividades y Calendarizacion

Actividad	Semana(s)	Responsable
Recopilacion y preprocesamiento del dataset BraTS2021	1-2	Equipo
Diseno e implementacion de la arquitectura U-Net	3-4	Equipo
Entrenamiento y validacion del modelo en GPU	5-6	Equipo
Implementacion del modulo de reglas de produccion	7	Equipo
Desarrollo de la interfaz grafica (app.py)	8-9	Equipo
Pruebas de integracion y correccion de errores	10	Equipo
Documentacion y elaboracion del documento Springer	11-12	Equipo
Presentacion final y entrega del proyecto	13	Equipo

Tabla 3. Calendarizacion de actividades del proyecto.

15 Referencias

- [1] Menze, B.H., et al.: The multimodal brain tumor image segmentation benchmark (BRATS). *IEEE Trans. Med. Imaging* 34(10), 1993-2024 (2015).
- [2] Ronneberger, O., Fischer, P., Brox, T.: U-Net: Convolutional networks for biomedical image segmentation. In: *MICCAI 2015, LNCS*, vol. 9351, pp. 234-241. Springer, Cham (2015).
- [3] Baid, U., et al.: The RSNA-ASNR-MICCAI BraTS 2021 benchmark on brain tumor segmentation and radiogenomic classification. *arXiv:2107.02314* (2021).
- [4] Buchanan, B.G., Shortliffe, E.H.: *Rule-Based Expert Systems: The MYCIN Experiments*. Addison-Wesley, Reading (1984).
- [5] Stupp, R., et al.: Radiotherapy plus concomitant and adjuvant temozolomide for glioblastoma. *N. Engl. J. Med.* 352(10), 987-996 (2005).
- [6] Isensee, F., et al.: nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation. *Nat. Methods* 18, 203-211 (2021).
- [7] Russell, S., Norvig, P.: *Artificial Intelligence: A Modern Approach*, 4th edn. Pearson, Hoboken (2020).
- [8] Clancey, W.J.: The epistemology of a rule-based expert system: a framework for explanation. *Artif. Intell.* 20(3), 215-251 (1983).

16 Glosario de Terminos

BraTS2021: Brain Tumor Segmentation Challenge 2021. Conjunto de datos publico con imagenes MRI de tumores cerebrales anotadas por expertos medicos.

Dice Score: Metrica de similitud entre dos mascaras binarias. Valor entre 0 y 1, donde 1 es coincidencia perfecta. Estandar en segmentacion medica.

GPU: Graphics Processing Unit. Tarjeta grafica utilizada para acelerar calculos paralelos necesarios en el entrenamiento de redes neuronales.

MRI: Magnetic Resonance Imaging (Resonancia Magnetica). Tecnica de imagen medica que utiliza campos magneticos para obtener imagenes del interior del cuerpo.

NIFTI: Neuroimaging Informatics Technology Initiative. Formato de archivo estandar para almacenar imagenes de neuroimagen (.nii, .nii.gz).

PyTorch: Biblioteca de aprendizaje profundo de codigo abierto desarrollada por Meta AI. Permite construir y entrenar redes neuronales en Python.

Red neuronal U-Net: Arquitectura de red convolucional con estructura simetrica encoder-decoder y conexiones de salto. Diseñada especificamente para segmentacion de imagenes medicas.

Regla de produccion: Estructura del conocimiento de la forma SI condicion ENTONCES accion, usada en sistemas expertos para razonamiento automatico.

Sistema experto: Programa de inteligencia artificial que imita la toma de decisiones de un especialista humano en un dominio especifico.

Slice axial: Corte horizontal del volumen cerebral en 3D. Cada volumen MRI contiene decenas de slices axiales apilados verticalmente.

T1ce: Modalidad MRI T1 con contraste de gadolinio. Maximiza el contraste entre tejido tumoral realzado y tejido sano circundante.

Tkinter: Libreria estandar de Python para creacion de interfaces graficas de usuario (GUI) en escritorio.

17 Anexos

Anexo A — Estructura del Proyecto

```
ProyectoIA2/
├── analisis/
│   ├── analizador.py    # Reglas de produccion clinicas
│   └── predictor.py     # Inferencia U-Net + seleccion de slice
├── entrenamiento/
│   ├── dataset_loader.py # Carga y preprocesamiento BraTS2021
│   └── entrenar.py       # Loop de entrenamiento con early stopping
├── modelos/
│   ├── modelo_unet.py    # Arquitectura U-Net 4 niveles
│   └── modelo_tumores.pth # Pesos entrenados
├── reportes/
│   └── reporte.py        # Generacion de PDF clinico
├── app.py               # Interfaz grafica principal
├── main.py              # Alternativa de consola
└── requirements.txt
```

Anexo B — Reglas de Produccion Completas

Las cinco reglas de produccion implementadas en analizador.py en notacion canonica:

```
R1: SI sum(mask_slice > 0) >= 100
    ENTONCES incluir_slice_en_entrenamiento()

R2: SI probabilidad_pixel > 0.5
    ENTONCES pixel_es_tumor = 1
    SINO      pixel_es_tumor = 0

R3: SI area_px > 0
    ENTONCES calcular(diametro, volumen_3D, hemisferio, confianza)

R4: SI diametro_cm < 2.0          ENTONCES riesgo = "BAJO"
    SI 2.0 <= diametro_cm <= 5.0 ENTONCES riesgo = "MODERADO"
    SI diametro_cm > 5.0          ENTONCES riesgo = "ALTO (CRITICO)"

R5: SI suma_pixeles_derecha > suma_pixeles_izquierda
    ENTONCES hemisferio = "IZQUIERDO" // Convencion radiologica
    SINO      hemisferio = "DERECHO"
```


Anexo C — Librerías y Dependencias

El archivo requirements.txt del proyecto contiene las siguientes dependencias:

```
torch>=2.0.0          # Red neuronal U-Net
torchvision>=0.15.0   # Utilidades de entrenamiento
nibabel>=5.0.0        # Carga de archivos NIfTI
opencv-python>=4.8.0  # Procesamiento de imagen
numpy>=1.24.0         # Operaciones matriciales
Pillow>=9.0.0         # Imagen para Tkinter
matplotlib>=3.7.0     # Visualizacion de resultados
scikit-learn>=1.3.0   # train_test_split
fpdf2>=2.7.0          # Generacion de reportes PDF
torch>=2.0.0          # Red neuronal U-Net, entrenamiento, inferencia
torchvision>=0.15.0   # Utilidades de vision (transformaciones, datasets)
nibabel>=5.0.0        # Carga de archivos NIfTI (.nii / .nii.gz)
opencv-python>=4.8.0  # Redimensionamiento, rotacion, flip (augmentacion)
numpy>=1.24.0         # Operaciones matriciales sobre slices y mascarar
Pillow>=9.0.0         # Conversion imagen → Tkinter (GUI)
matplotlib>=3.7.0     # Panel de resultados (3 paneles MRI/seg/metricas)
scikit-learn>=1.3.0   # train_test_split para split train/val
fpdf2>=2.7.0          # Reporte PDF clinico automatico
```